

First Sprint Report 28/05/2026 to 04/06/2026

Final report of the Group 15

Group members: Francesco Di Crescenzo

Nicola Alessandro Galluzzo

Carlo Maria Franchetti

Luca De Iuliis

Step 1: Preparation

Trello Board: <https://trello.com/b/ofbNyj2w/music-playlist-manager-group-15>

Member chosen as Product Owner: Nicola Alessandro Galluzzo

User stories produced in this phase:

- US-01 Creazione della traccia
- US-02 Visualizzazione delle tracce
- US-04 Eliminazione di una Traccia
- US-05 Creazione di una playlist
- US-06 Aggiunta di una traccia ad una playlist
- US-07 Rimozione traccia da una playlist
- US-08 Visualizzazione playlist
- US-10 Eliminazione playlist

Definition of Done:

Una User Story si considera DONE solo quando soddisfa TUTTI i criteri obbligatori (✓) elencati di seguito, oltre ai propri Acceptance Criteria specifici. I criteri indicati da questo simbolo (○) si considerano opzionali.

Codice

- 1. Compilazione pulita: Il codice compila senza errori e i warning vengono gestiti in maniera appropriata ✓
- 2. Naming conventions: Le naming conventions Java sono rispettate ✓
- 3. Version control: Il codice è stato caricato (committato e pushato) sul branch principale del repository condiviso ✓
- 4. Commenti significativi: I commenti descrivono il "perché" delle scelte non ovvie, non ridondano il codice ✓

Design

- 5. Alta coesione: Alta coesione tra responsabilità delle classi ✓
- 6. Basso accoppiamento: Basso accoppiamento tra classi/package ✓
- 7. Astrazione: Uso di interfacce o classi astratte ove necessario ○
- 8. Principio DRY: Assenza di duplicazioni (Don't Repeat Yourself) ✓

- 9. Rispetto dell'architettura: Le modifiche rispettano l'architettura definita e non introducono violazioni dei layer/package ✓

Test

- 10. Verifica dei criteri: Ogni Acceptance Criterion della User Story è stato verificato con successo (tramite test automatizzati JUnit per la logica di backend e test manuali per l'interfaccia UI) ✓
- 11. Copertura scenari: Copertura dei Casi Limite essenziali: i test includono il percorso ideale e i principali scenari di errore previsti dalla card ✓
- 12. Test superati: Passaggio dei Test: tutti i test passano prima dell'integrazione nel branch principale ✓

Processo SCRUM

- 13. Integrazione codice: Codice integrato tramite merge/review nel repository GitHub ✓
- 14. Code Review: Revisione del codice effettuata da almeno un altro membro ✓
- 15. Aggiornamento board: Task spostato in Done su Trello ✓
- 16. Artefatti Scrum: Aggiornamento degli artefatti Scrum quando necessario ○ Interfaccia Utente
- 17. Aggiornamento dinamico UI: L'interfaccia aggiorna correttamente lo stato senza riavvio ✓
- 18. Feedback visivo: Messaggi di errore comprensibili e visibili ✓

Language/Development Environment:

Il progetto è sviluppato su IntelliJ IDEA usando la versione 17 di Java.

Phase 2.1: First Sprint - Planning

Project Velocity Initial Estimate: 10 Story Point.

User stories chosen for this Sprint:

- US-01 Creazione della traccia
- US-02 Visualizzazione delle tracce
- US-04 Eliminazione di una Traccia
- US-05 Creazione di una playlist
- US-08 Visualizzazione playlist
- US-10 Eliminazione playlist

Phase 2.2: First Sprint - Development

Tasks assigned to Team members:

	Francesco Di Crescenzo	Nicola Alessandro Galluzzo	Carlo Maria Franchetti	Luca De Iuliis
--	------------------------	----------------------------	------------------------	----------------

Assigned and completed tasks	Setup iniziale del progetto Implementazione del package Model e scrittura del codice delle classi: Track, Playlist, MusicLibrary. Implementazione e scrittura del codice delle classi: Laucher, MainPlayerApplication Implementazione del package Service e scrittura del codice delle classi: MusicLibraryService, PlaylistService, TrackService. Implementazione del package Util e scrittura del codice delle classi: AlertUtil. Implementazione del package Controller e scrittura del codice delle classi: MusicPlayerController. Implementazione del package	Gestione documentazione Creazione burndown chart Sviluppo materiale presentazione 1st Sprint	Gestione dei test delle classi model Sviluppo classi service(Track e Playlist) Responsabile dell'architettura software Gestione documentazione 1st Sprint	Creazione classe Playlist Creazione e ordinamento dei test JUnit (TrackTest, PlaylistTest, MusicLibraryTest, TrackServiceTest, PlaylistServiceTest, MusicLibraryServiceTest) Creazione e gestione dello Sprint Report Creazione Diagrammi di Classe e Sequenza
------------------------------	---	---	---	--

	Exception e scrittura del codice delle classi: PlaylistValidationException TrackValidationException. Implementazione e Interfaccia grafica e scrittura del codice di: player-view.fxml Gestione Trello e User Story. Gestione Interfaccia Tramite Scene Builder e JavaFX			
Tasks assigned but not completed	\	\	\	\
Tasks not assigned to any team member	\			\

User stories added or edited by the Product Owner:

ADDED:

- US-06 Aggiunta di una traccia ad una playlist
- US-07 Rimozione traccia da una playlist

Phase 2.3: First Sprint - Review

Full user stories at the end of this Sprint:

- US-01 Creazione della traccia
- US-02 Visualizzazione delle tracce
- US-04 Eliminazione di una Traccia
- US-05 Creazione di una playlist
- US-06 Aggiunta di una traccia ad una playlist

- US-07 Rimozione traccia da una playlist
- US-08 Visualizzazione playlist
- US-10 Eliminazione playlist

Any user stories rejected by the Product Owner:

- Nessuna.

Project Velocity measured: 13 Story Point.

Phase 2.4: First Sprint - Retrospective

The decisions taken in the retrospective meeting of this Sprint are:

Stop	Less of	Keep doing	More of	Start
· Modifiche frequenti a chi deve fare un task assegnato. · Fare commit senza essere sicuri che il codice compili correttamente.	· Modifiche al codice senza testarlo · Divisione irregolare del lavoro.	· Mantenere una comunicazione costante tra i membri del team. · Aggiornamento regolare del repository Git. · Mantenere aggiornata la board di Trello.	· Verifiche intermedie dello stato di avanzamento. · Applicare sistematicamente le attività di testing previste dalla Definition of Done.	· Effettuare code review prima del merge. · Definire meglio le stime dei task durante la Sprint Planning. · Iniziare a fare il Backlog Grooming (raffinamento del backlog) all'inizio dello Sprint per chiarire meglio le User Story future.